UNIVERSIDAD DE LA GUAJIRA
Facultad de Ingenierías · Ingeniería de Sistemas

Parcial 3 · Ingeniería de Software I

Equipo de calidad y mantenimiento de EnvíaYa · Tercer corte · Unidades 6 y 7

Estudiante	Carlos David Sierra	**Código estudiantil**	0182410125
Asignatura	Ingeniería de Software I (273365)	**Grupo**	B1 · VI semestre
Docente	Ing. Robert Damián Quintero Laverde	**Fecha**	11 de noviembre de 2026
Repositorio	https://github.com/calo159/EnviaYa		

Mis parámetros

A	B	C	D	E	F	PMAX	L1	L2	RR	RN	V	S	CAMBIO
4	1	0	1	2	5	50	3	10	20 %	50 %	15 %	$2.000	0

Código 0182410125 → seis últimos dígitos 4 1 0 1 2 5. Detalle del cálculo en la Parte 1.

Parte 1 · Requisitos y criterios de aceptación

1.1 Mis parámetros (código 0182410125)

Seis últimos dígitos: **4 1 0 1 2 5** → A=4, B=1, C=0, D=1, E=2, F=5.

A	B	C	D	E	F	PMAX	L1	L2	RR	RN	V	S	CAMBIO
4	1	0	1	2	5	50 kg	3 kg	10 kg	20 %	50 %	15 %	$2.000	0

Cálculo:

- PMAX = 30 + 5·4 = **50**
- L1 = 2 + (1 mod 3) = **3**
- L2 = 10 + 0 = **10**
- RR = 10 + 5·(2 mod 4) = 10 + 10 = **20 %**
- RN = 40 + 5·(5 mod 3) = 40 + 10 = **50 %**
- V = 10 + 5·(4 mod 3) = 10 + 5 = **15 %**
- S = 2000 + 500·(0 mod 4) = **$2.000**
- CAMBIO = (A + F) mod 3 = (4 + 5) mod 3 = 9 mod 3 = **0** → nueva zona insular (San Andrés), recargo 70 %, sin cupón.

Reglas de paridad:

Dígito	Valor	Paridad	Regla resultante
D (tramo 1)	1	impar	Tramo 1 **no incluye** el límite: peso **< 3**
E (fin de semana)	2	par	El recargo se suma **antes** de los descuentos
B (cliente y cupón)	1	impar	Se aplican **ambos**: primero el del cliente y luego el cupón
F (cupón inexistente)	5	impar	Un cupón que no existe es **error**

1.2 Especificación completa (R1–R8) con mis valores

Entradas: `peso` (kg, admite decimales), `zona` (urbana, rural, nacional), `cliente` (normal, frecuente, vip), `dia` (1 a 7; 6 = sábado, 7 = domingo) y `cupon` (opcional).

ID	Regla
R1	El peso debe cumplir 0 < peso ≤ 50. Si no, se lanza el error "peso inválido".
R2	Si la zona no es urbana, rural ni nacional, o el día no está entre 1 y 7, se lanza un error.
R3	Tarifa base. Tramo 1 (peso < 3): $6.000. Tramo 2 (3 ≤ peso ≤ 10): $9.000. Más de 10 kg: $9.000 + $800 por cada kg sobre 10, proporcional.
R4	Recargo por zona. Rural: +20 %. Nacional: +50 %. Urbana: sin recargo.
R5	Fin de semana (día 6 o 7): +$2.000, **antes** de los descuentos.
R6	Descuentos. Cliente frecuente: 5 %. Cliente VIP: 15 %. Cupón ENVIO10: 10 %. Si hay cliente con descuento y cupón se aplican **ambos**: primero el del cliente y luego el cupón (sobre el valor ya descontado).
R7	Un cupón que no existe es **error** (el único cupón válido es `ENVIO10`).

ID	Regla
R8	El costo nunca es menor que $5.000. El resultado se redondea al múltiplo de $100 más cercano; los $50 redondean hacia arriba.

Orden de cálculo: base (R3) → zona (R4) → fin de semana (R5) → descuento de cliente → cupón (R6) → mínimo $5.000 → redondeo (R8).

Decisiones de interpretación (la especificación no las fija; se documentan):

- El mínimo de $5.000 se aplica antes del redondeo.
- Un cliente que no es normal, frecuente ni vip no tiene regla de error en R1–R8; se trata como normal. Queda como pregunta abierta para el negocio.
- `cupon = None` significa "sin cupón". Cualquier otra cadena distinta de `ENVIO10` (incluido `envio10` en minúscula o cadena vacía) es un cupón inexistente.

1.3 Historias de usuario

HU-1 · Validar el envío (R1, R2, R7) Como operador de EnvíaYa quiero que el sistema rechace envíos con datos inválidos para no cobrar ni despachar envíos imposibles.

- *Dado* un peso de 0 kg o negativo, *cuando* calculo el costo, *entonces* se lanza "peso inválido".
- *Dado* un peso de 50,01 kg, *cuando* calculo el costo, *entonces* se lanza "peso inválido"; con exactamente 50 kg se calcula normalmente.
- *Dado* una zona "insular" (o cualquier valor fuera de urbana, rural y nacional), *cuando* calculo el costo, *entonces* se lanza un error.
- *Dado* un día 0 u 8, *cuando* calculo el costo, *entonces* se lanza un error.
- *Dado* un cupón "DESCUENTO99", *cuando* calculo el costo, *entonces* se lanza un error (no se ignora).

HU-2 · Calcular la tarifa con recargos (R3, R4, R5) Como cliente quiero conocer el costo según peso, zona y día para saber cuánto pagaré.

- *Dado* un peso de 2,99 kg, *cuando* calculo, *entonces* la base es $6.000; con 3 kg la base es $9.000.
- *Dado* un peso de 10 kg, *cuando* calculo, *entonces* la base es $9.000; con 10,5 kg es $9.400.
- *Dado* una zona rural o nacional, *cuando* calculo, *entonces* se suma 20 % o 50 % a la base.
- *Dado* un día sábado (6) o domingo (7), *cuando* calculo, *entonces* se suman $2.000 antes de aplicar descuentos; de lunes a viernes no se suma.

HU-3 · Aplicar descuentos, mínimo y redondeo (R6, R8) Como cliente frecuente, VIP o con cupón quiero que se apliquen mis descuentos para pagar menos, y como negocio quiero un costo mínimo y un redondeo claro.

- *Dado* un cliente frecuente o VIP, *cuando* calculo, *entonces* se descuenta 5 % o 15 %.
- *Dado* un cliente VIP con cupón ENVIO10, *cuando* calculo, *entonces* se aplican ambos: primero 15 % y luego 10 % sobre el resultado.
- *Dado* un valor calculado menor a $5.000, *cuando* calculo, *entonces* el costo es $5.000.
- *Dado* un valor de $8.550, *cuando* calculo, *entonces* el resultado es $8.600 (los $50 suben); con $8.549 es $8.500.

1.4 Matriz de trazabilidad (inicial)

La columna de casos de prueba se completa con el detalle de la Parte 3; la de defectos, en la Parte 5.

Requisito	Historia	Casos de prueba	Defectos asociados
R1	HU-1	CP-01, CP-02, CP-03, CP-04, CP-05, CP-06	—
R2	HU-1	CP-07, CP-08, CP-09	DEF-02, DEF-03
R3	HU-2	CP-10, CP-11, CP-12, CP-13, CP-14, CP-15, CP-16	DEF-01
R4	HU-2	CP-17, CP-18, CP-19	—
R5	HU-2	CP-20, CP-21, CP-22, CP-23	DEF-04
R6	HU-3	CP-24, CP-25, CP-26, CP-27, CP-28	—
R7	HU-1	CP-29, CP-30	DEF-05
R8	HU-3	CP-31, CP-32, CP-33, CP-34	DEF-06

Parte 2 · Plan de pruebas

2.1 Alcance

Dentro del alcance

- La función de cálculo de costo de envío de `envios.py` (`calc`), versión heredada 1.0.0 y sus versiones corregidas.
- Las reglas R1 a R8 con mis parámetros: PMAX = 50, L1 = 3 (no incluido), L2 = 10, rural 20 %, nacional 50 %, VIP 15 %, fin de semana $2.000 antes de descuentos, descuentos acumulados (cliente y luego cupón), cupón inexistente = error.
- La solicitud de cambio 0 (zona insular), una vez implementada.

Fuera del alcance

- Interfaz de usuario, base de datos, facturación y cualquier otro módulo de EnvíaYa: no existen en el código entregado.
- Pruebas de rendimiento, carga y seguridad: la función es un cálculo puro sin entrada/salida.
- Validación de tipos (por ejemplo, peso como texto): la especificación no la define.

2.2 Niveles de prueba y cómo se aplican a este caso

Nivel	Cómo se aplica aquí
Unidad	Es el nivel principal. Se prueba `calc` de forma aislada con pytest: particiones, valores límite de peso (0, 3, 10, 50), tabla de decisión y caminos independientes. Tras la refactorización también se prueban por separado las funciones extraídas (tarifa base, redondeo) y cada estrategia de zona.
Integración	En la versión heredada hay una sola función, así que no hay nada que integrar. Después de refactorizar sí: se verifica que `calc` combine bien tarifa base + estrategia de zona + recargo de fin de semana + descuentos + redondeo. Los casos combinados (por ejemplo CP-23, CP-27, CP-28 y CP-36) cumplen ese papel porque atraviesan todas las piezas en orden.
Sistema	Se prueba el módulo completo como caja negra desde su interfaz pública (`from envios import calc`), ejecutando toda la suite en un ambiente limpio y comprobando R1–R8 de punta a punta, incluidos los errores. En un EnvíaYa real este nivel incluiría la pantalla de cotización y el servicio que llama a la función.
Aceptación	Los criterios Dado/Cuando/Entonces de HU-1, HU-2 y HU-3 se ejecutan como pruebas de aceptación. El "cliente" (el docente, en la sustentación) valida que los resultados coinciden con los calculados a mano en la Parte 3.

2.3 Estrategia

1. **Diseño de caja negra** a partir de la especificación: particiones de equivalencia, análisis de valores límite y tabla de decisión (Parte 3).
2. **Diseño de caja blanca** sobre el código heredado: grafo de flujo, complejidad ciclomática y caminos independientes (Parte 4).
3. **Automatización** de todos los casos con pytest. Los resultados esperados se calculan a mano desde la especificación, nunca ejecutando el código.
4. **Ejecución contra el código heredado** (v1.0.0) para detectar defectos; cada fallo se analiza y se registra (Parte 5).
5. **Corrección** de cada defecto en su rama, y **regresión**: la misma suite completa se ejecuta después de cada corrección y después de la refactorización.

6. **Cobertura** de sentencias y ramas con pytest-cov; meta: 100 % de sentencias y ≥ 95 % de ramas.
7. Para la solicitud de cambio se usa **TDD**: primero las pruebas (rojo), luego el código (verde).

2.4 Ambiente

Elemento	Valor
Sistema operativo	Windows 11 Pro
Lenguaje	Python 3.13
Framework de pruebas	pytest 9.2
Cobertura	pytest-cov 7.1 (coverage 7.17), con medición de ramas
Complejidad	radon 6.0
Control de versiones	Git 2.54 + GitHub (`calo159/EnviaYa`)
Datos de prueba	Valores escritos en los propios casos; no hay datos externos

Comando de ejecución: `python -m pytest --cov=envios --cov-branch --cov-report=term-missing`

2.5 Roles

Trabajo individual; una sola persona asume todos los roles.

Rol	Responsabilidad	Quién
Analista de pruebas	Especificación, historias, diseño de casos	Carlos David Sierra
Automatizador	Scripts de pytest y medición de cobertura	Carlos David Sierra
Desarrollador de mantenimiento	Correcciones y refactorización	Carlos David Sierra
Gestor de configuración	Ramas, etiquetas, CHANGELOG	Carlos David Sierra
Cliente / aprobador	Aceptación en la sustentación	Docente

2.6 Riesgos y mitigación

#	Riesgo	Prob.	Impacto	Mitigación
1	Resultado esperado mal calculado a mano (una prueba "correcta" que valida algo errado)	Media	Alto	Mostrar el cálculo paso a paso; revisar cada caso que falle antes de declararlo defecto.
2	Parámetros personalizados mal derivados del código estudiantil	Baja	Muy alto (nota 0)	Tabla de parámetros con las fórmulas resueltas al inicio; `CONFIG` verificado contra ella.
3	La refactorización introduce defectos nuevos	Media	Alto	Suite de regresión completa antes y después; cambios pequeños en ramas separadas.
4	Errores de precisión con decimales (pesos como 10,01 o descuentos encadenados) que alteran el redondeo	Media	Medio	Casos en los límites del redondeo ($49, $50, $51); usar aritmética decimal exacta en la versión corregida.
5	Ambigüedad de la especificación (cliente inválido, orden mínimo/redondeo)	Media	Medio	Documentar la interpretación adoptada en la Parte 1 y marcarla como pregunta abierta.
6	Tiempo limitado (un solo día)	Alta	Medio	Priorizar por riesgo: primero límites y reglas que dependen de la paridad; commits frecuentes.
7	Conflictos al integrar varias ramas de corrección sobre el mismo archivo	Media	Bajo	Una rama por defecto, integradas una a una y con la suite ejecutada tras cada merge.

2.7 Criterios de entrada

- Especificación R1–R8 escrita con mis parámetros y `CONFIG` diligenciado.
- Código heredado bajo control de versiones y etiquetado como `v1.0.0`.
- Ambiente instalado (Python, pytest, pytest-cov) y la suite ejecutable.
- Casos de prueba diseñados, con resultado esperado calculado a mano.

2.8 Criterios de salida

- 100 % de los casos diseñados ejecutados.
- 100 % de los casos pasan sobre la versión corregida.
- Ningún defecto abierto de severidad crítica o alta; todo defecto registrado está corregido y verificado con su caso.
- Cobertura de sentencias = 100 % y de ramas ≥ 95 %.
- Matriz de trazabilidad completa: cada requisito tiene al menos un caso que pasa.
- Versión nueva etiquetada y CHANGELOG actualizado.

Criterios de suspensión: si la suite no puede ejecutarse (error de importación o de ambiente) se suspenden las pruebas hasta corregir el ambiente.

Parte 3 · Diseño de casos de caja negra

Todos los resultados esperados salen de mi especificación (Parte 1), calculados a mano. Ninguno se obtuvo ejecutando el código.

3.1 Particiones de equivalencia

Entrada	Clase	Tipo	Partición	Valor representativo
peso	PE-P1	Inválida	peso ≤ 0	−0,01 ; 0
peso	PE-P2	Válida	0 < peso < 3 (tramo 1)	2
peso	PE-P3	Válida	3 ≤ peso ≤ 10 (tramo 2)	5
peso	PE-P4	Válida	10 < peso ≤ 50 (tramo 3)	25
peso	PE-P5	Inválida	peso > 50	50,01
zona	PE-Z1	Válida	urbana	"urbana"
zona	PE-Z2	Válida	rural	"rural"
zona	PE-Z3	Válida	nacional	"nacional"
zona	PE-Z4	Inválida	cualquier otro valor	"insular"
cliente	PE-C1	Válida	normal	"normal"
cliente	PE-C2	Válida	frecuente	"frecuente"
cliente	PE-C3	Válida	vip	"vip"
día	PE-D1	Inválida	día < 1	0
día	PE-D2	Válida	1 a 5 (entre semana)	1 ; 5
día	PE-D3	Válida	6 y 7 (fin de semana)	6 ; 7
día	PE-D4	Inválida	día > 7	8
cupón	PE-K1	Válida	sin cupón	None
cupón	PE-K2	Válida	cupón existente	"ENVIO10"
cupón	PE-K3	Inválida	cupón inexistente (F impar → error)	"DESCUENTO99" ; "envio10"

3.2 Valores límite de peso

Mi L1 = 3 **no está incluido** en el tramo 1 (D = 1, impar): 2,99 es tramo 1 y 3 ya es tramo 2. L2 = 10 sí está incluido en el tramo 2. PMAX = 50 es válido.

Límite	Valor	Clase	Resultado esperado (urbana, normal, día 1, sin cupón)	Caso
0	−0,01	inválido	error “peso inválido”	CP-02
0	0	inválido	error “peso inválido”	CP-01
0	0,01	tramo 1	$6.000	CP-03
L1 = 3	2,99	tramo 1	$6.000	CP-10
L1 = 3	3	tramo 2	$9.000	CP-11
L1 = 3	3,01	tramo 2	$9.000	CP-12
L2 = 10	9,99	tramo 2	$9.000	CP-13

Límite	Valor	Clase	Resultado esperado (urbana, normal, día 1, sin cupón)	Caso
L2 = 10	10	tramo 2	$9.000	CP-14
L2 = 10	10,01	tramo 3	9.000 + 0,01·800 = 9.008 → $9.000	CP-15
PMAX = 50	49,99	tramo 3	9.000 + 39,99·800 = 40.992 → $41.000	CP-06
PMAX = 50	50	tramo 3	9.000 + 40·800 = $41.000	CP-04
PMAX = 50	50,01	inválido	error "peso inválido"	CP-05

3.3 Tabla de decisión: cliente × cupón × fin de semana

Reglas que aplican: **B impar** → se aplican ambos descuentos, primero cliente y luego cupón. **E par** → el recargo de $2.000 se suma antes de los descuentos.

Condiciones fijas para calcular: peso = 5 kg (base $9.000), zona urbana. Con fin de semana el valor antes de descuentos es 9.000 + 2.000 = 11.000.

Regla	Cliente	Cupón	Fin de semana	+$2.000 antes	Desc. cliente	Desc. cupón 10 %	Cálculo	Esperado
TD-01	normal	no	no	—	—	—	9.000	$9.000
TD-02	normal	no	sí	✓	—	—	11.000	$11.000
TD-03	normal	ENVIO10	no	—	—	✓	9.000 × 0,90 = 8.100	$8.100
TD-04	normal	ENVIO10	sí	✓	—	✓	11.000 × 0,90 = 9.900	$9.900
TD-05	frecuente	no	no	—	5 %	—	9.000 × 0,95 = 8.550	$8.600
TD-06	frecuente	no	sí	✓	5 %	—	11.000 × 0,95 = 10.450	$10.500
TD-07	frecuente	ENVIO10	no	—	5 %	✓	8.550 × 0,90 = 7.695	$7.700
TD-08	frecuente	ENVIO10	sí	✓	5 %	✓	10.450 × 0,90 = 9.405	$9.400
TD-09	vip	no	no	—	15 %	—	9.000 × 0,85 = 7.650	$7.700
TD-10	vip	no	sí	✓	15 %	—	11.000 × 0,85 = 9.350	$9.400
TD-11	vip	ENVIO10	no	—	15 %	✓	7.650 × 0,90 = 6.885	$6.900
TD-12	vip	ENVIO10	sí	✓	15 %	✓	9.350 × 0,90 = 8.415	$8.400

Cupón inexistente: para cualquier cliente y día la acción es **error** (regla F impar); se cubre con CP-29 y CP-30.

3.4 Matriz de casos de prueba

Salvo que se indique otra cosa: zona urbana, cliente normal, día 1 (lunes), sin cupón. Técnicas: PE = partición de equivalencia, VL = valor límite, TD = tabla de decisión.

ID	Req.	peso	zona	cliente	día	cupón	Resultado esperado	Técnica
CP-01	R1	0	urbana	normal	1	—	Error "peso inválido"	VL
CP-02	R1	−0,01	urbana	normal	1	—	Error "peso inválido"	VL / PE-P1
CP-03	R1, R3	0,01	urbana	normal	1	—	$6.000	VL
CP-04	R1, R3	50	urbana	normal	1	—	$41.000	VL
CP-05	R1	50,01	urbana	normal	1	—	Error "peso inválido"	VL / PE-P5
CP-06	R1, R3, R8	49,99	urbana	normal	1	—	$41.000	VL
CP-07	R2	5	insular	normal	1	—	Error (zona inválida)	PE-Z4
CP-08	R2	5	urbana	normal	0	—	Error (día inválido)	VL / PE-D1
CP-09	R2	5	urbana	normal	8	—	Error (día inválido)	VL / PE-D4
CP-10	R3	2,99	urbana	normal	1	—	$6.000	VL
CP-11	R3	3	urbana	normal	1	—	$9.000	VL
CP-12	R3	3,01	urbana	normal	1	—	$9.000	VL
CP-13	R3	9,99	urbana	normal	1	—	$9.000	VL
CP-14	R3	10	urbana	normal	1	—	$9.000	VL
CP-15	R3, R8	10,01	urbana	normal	1	—	$9.000	VL
CP-16	R3	25	urbana	normal	1	—	$21.000	PE-P4
CP-17	R4	5	urbana	normal	1	—	$9.000	PE-Z1
CP-18	R4	5	rural	normal	1	—	$10.800	PE-Z2
CP-19	R4	5	nacional	normal	1	—	$13.500	PE-Z3
CP-20	R5	5	urbana	normal	5	—	$9.000	VL día
CP-21	R5	5	urbana	normal	6	—	$11.000	VL día / PE-D3
CP-22	R5	5	urbana	normal	7	—	$11.000	VL día / PE-D3
CP-23	R4, R5, R6	5	rural	vip	6	—	$10.900	TD (regla E)
CP-24	R6	2	urbana	frecuente	1	—	$5.700	PE-C2
CP-25	R6	2	urbana	vip	1	—	$5.100	PE-C3
CP-26	R6	5	urbana	normal	1	ENVIO10	$8.100	PE-K2
CP-27	R4, R6, R8	5	nacional	vip	1	ENVIO10	$10.300	TD (regla B)
CP-28	R4, R5, R6, R8	5	rural	frecuente	7	ENVIO10	$10.900	TD (reglas B y E)
CP-29	R7	5	urbana	normal	1	DESCUENTO99	Error (cupón inexistente)	PE-K3
CP-30	R7	5	urbana	vip	6	envio10	Error (cupón inexistente)	PE-K3
CP-31	R8	1	urbana	vip	1	ENVIO10	$5.000	VL mínimo
CP-32	R8	5	urbana	frecuente	1	—	$8.600	VL redondeo ($50)

ID	Req.	peso	zona	cliente	día	cupón	Resultado esperado	Técnica
CP-33	R8	5	urbana	frecuente	1	ENVIO10	$7.700	VL redondeo ($95)
CP-34	R8	10,06	urbana	normal	1	—	$9.000	VL redondeo ($48)
CP-35	R6, R8	1	urbana	frecuente	1	ENVIO10	$5.100	VL mínimo
CP-36	R3–R8	20	nacional	frecuente	7	ENVIO10	$23.500	Combinado
TD-01 … TD-12	R5, R6, R8	5	urbana	(tabla)	1 o 6	(tabla)	Ver tabla 3.3	TD

Total: **36 casos CP + 12 casos de la tabla de decisión = 48 casos**.

Cálculos cortos de los casos que no se detallan abajo:

- CP-16: 9.000 + (25 − 10)·800 = 9.000 + 12.000 = 21.000.
- CP-18: 9.000 × 1,20 = 10.800. CP-19: 9.000 × 1,50 = 13.500.
- CP-21 y CP-22: 9.000 + 2.000 = 11.000.
- CP-24: 6.000 × 0,95 = 5.700. CP-25: 6.000 × 0,85 = 5.100. CP-26: 9.000 × 0,90 = 8.100.
- CP-28: 9.000 × 1,20 = 10.800; + 2.000 = 12.800; × 0,95 = 12.160; × 0,90 = 10.944 → 10.900.
- CP-33: 9.000 × 0,95 = 8.550; × 0,90 = 7.695 → 7.700.
- CP-34: 9.000 + 0,06·800 = 9.048 → 9.000.
- CP-35: 6.000 × 0,95 = 5.700; × 0,90 = 5.130 (≥ 5.000) → 5.100.

3.5 Cálculo paso a paso (6 casos)

CP-11 · peso = 3 kg, urbana, normal, día 1, sin cupón 1. Tramo: D = 1 es impar, el tramo 1 es peso < 3. Como 3 no es menor que 3, va al tramo 2 → $9.000 2. Zona urbana: sin recargo → $9.000 3. Lunes: sin recargo de fin de semana → $9.000 4. Cliente normal y sin cupón: sin descuentos → $9.000 5. Es mayor que $5.000 y ya es múltiplo de $100 → **resultado esperado: $9.000**

CP-23 · peso = 5 kg, rural, vip, día 6 (sábado), sin cupón 1. Tramo: 3 ≤ 5 ≤ 10, tramo 2 → $9.000 2. Zona rural +20 % → 9.000 × 1,20 = $10.800 3. Sábado, y E = 2 es par: el recargo va antes de los descuentos → 10.800 + 2.000 = $12.800 4. VIP 15 %, sin cupón → 12.800 × 0,85 = $10.880 5. Es mayor que $5.000. Se redondea a $100 (80 ≥ 50, sube) → **resultado esperado: $10.900**

(Si el recargo fuera después: 10.800 × 0,85 = 9.180 + 2.000 = 11.180 → 11.200. El caso distingue la regla E.)

CP-27 · peso = 5 kg, nacional, vip, día 1, cupón ENVIO10 1. Tramo 2 → $9.000 2. Zona nacional +50 % → 9.000 × 1,50 = $13.500 3. Lunes: sin recargo → $13.500 4. B = 1 es impar: se aplican ambos. Primero VIP 15 % → 13.500 × 0,85 = $11.475; luego cupón 10 % → 11.475 × 0,90 = $10.327,50 5. Es mayor que $5.000. Se redondea a $100 (27,50 < 50, baja) → **resultado esperado: $10.300**

(Con "solo el mayor" daría 11.475 → 11.500. El caso distingue la regla B.)

CP-31 · peso = 1 kg, urbana, vip, día 1, cupón ENVIO10 1. Tramo: 1 < 3, tramo 1 → $6.000 2. Urbana: sin recargo → $6.000 3. Lunes: sin recargo → $6.000 4. VIP 15 % → 6.000 × 0,85 = $5.100; cupón 10 % → 5.100 × 0,90 = $4.590 5. 4.590 es menor que $5.000 → se cobra el mínimo → **resultado esperado: $5.000**

CP-32 · peso = 5 kg, urbana, frecuente, día 1, sin cupón 1. Tramo 2 → $9.000 2. Urbana: sin recargo → $9.000 3. Lunes: sin recargo → $9.000 4. Frecuente 5 %, sin cupón → 9.000 × 0,95 = $8.550 5. Es mayor que $5.000. Se

redondea a $100: los $50 redondean hacia arriba → **resultado esperado: $8.600**

CP-36 · peso = 20 kg, nacional, frecuente, día 7 (domingo), cupón ENVIO10 1. Tramo: 20 > 10, tramo 3 → 9.000 + (20 − 10) × 800 = $17.000 2. Zona nacional +50 % → 17.000 × 1,50 = $25.500 3. Domingo, recargo antes de los descuentos → 25.500 + 2.000 = $27.500 4. Ambos descuentos: frecuente 5 % → 27.500 × 0,95 = $26.125; cupón 10 % → 26.125 × 0,90 = $23.512,50 5. Es mayor que $5.000. Se redondea a $100 (12,50 < 50, baja) → **resultado esperado: $23.500**

3.6 Matriz de trazabilidad actualizada

Requisito	Historia	Casos de prueba
R1	HU-1	CP-01, CP-02, CP-03, CP-04, CP-05, CP-06
R2	HU-1	CP-07, CP-08, CP-09
R3	HU-2	CP-03, CP-04, CP-06, CP-10 a CP-16, CP-36
R4	HU-2	CP-17, CP-18, CP-19, CP-23, CP-27, CP-28, CP-36
R5	HU-2	CP-20, CP-21, CP-22, CP-23, CP-28, CP-36, TD-01 a TD-12
R6	HU-3	CP-23 a CP-28, CP-35, CP-36, TD-01 a TD-12
R7	HU-1	CP-29, CP-30
R8	HU-3	CP-06, CP-15, CP-31 a CP-36, TD-05 a TD-12

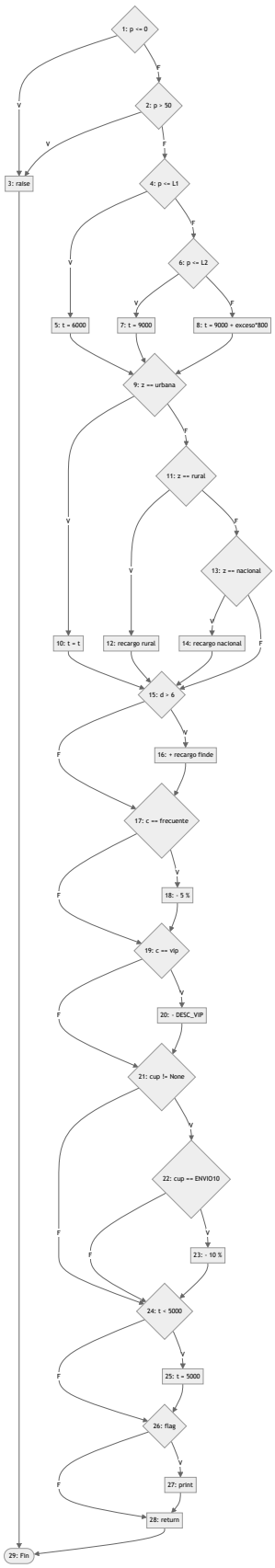
Parte 4 · Caja blanca

Análisis sobre el código heredado `envios.py` v1.0.0 (función `calc`, líneas 11 a 40).

4.1 Grafo de flujo

Cada nodo es una sentencia o condición simple. La condición compuesta de la línea 12 (`p <= 0 or p > 50`) se separa en dos nodos predicado (1 y 2) porque el `or` evalúa en cortocircuito.

Nodo	Línea	Contenido	Tipo
1	12	`p <= 0`	predicado
2	12	`p > 50`	predicado
3	13	`raise ValueError("peso invalido")`	sentencia
4	14	`p <= CONFIG["L1"]`	predicado
5	15	`t = 6000`	sentencia
6	16	`p <= CONFIG["L2"]`	predicado
7	17	`t = 9000`	sentencia
8	19	`t = 9000 + (p - L2) * 800`	sentencia
9	20	`z == "urbana"`	predicado
10	21	`t = t`	sentencia
11	22	`z == "rural"`	predicado
12	23	`t = t + t * RECARGO_RURAL`	sentencia
13	24	`z == "nacional"`	predicado
14	25	`t = t + t * RECARGO_NACIONAL`	sentencia
15	27	`d > 6`	predicado
16	28	`t = t + RECARGO_FINDE`	sentencia
17	29	`c == "frecuente"`	predicado
18	30	`t = t - t * 0.05`	sentencia
19	31	`c == "vip"`	predicado
20	32	`t = t - t * DESC_VIP`	sentencia
21	33	`cup != None`	predicado
22	34	`cup == "ENVIO10"`	predicado
23	35	`t = t - t * 0.10`	sentencia
24	36	`t < 5000`	predicado
25	37	`t = 5000`	sentencia
26	38	`flag`	predicado
27	39	`print("costo:", t)`	sentencia
28	40	`return int(t / 100) * 100`	sentencia
29	—	Fin	salida



Lista de aristas (42):

1→2, 1→3, 2→3, 2→4, 3→29, 4→5, 4→6, 5→9, 6→7, 6→8, 7→9, 8→9, 9→10, 9→11, 10→15, 11→12, 11→13, 12→15, 13→14, 13→15, 14→15, 15→16, 15→17, 16→17, 17→18, 17→19, 18→19, 19→20, 19→21, 20→21, 21→22, 21→24, 22→23, 22→24, 23→24, 24→25, 24→26, 25→26, 26→27, 26→28, 27→28, 28→29.

4.2 Complejidad ciclomática de tres formas

a) Aristas y nodos: V(G) = E − N + 2 = 42 − 29 + 2 = **15**

b) Nodos predicado: V(G) = P + 1. Los predicados son los nodos 1, 2, 4, 6, 9, 11, 13, 15, 17, 19, 21, 22, 24 y 26 → P = 14 → V(G) = 14 + 1 = **15**

c) Regiones: el grafo dibujado en el plano tiene 14 regiones cerradas (una por cada nodo predicado, que abre una bifurcación que luego se cierra) más la región exterior → **15** regiones.

Las tres formas coinciden: **V(G) = 15**. La herramienta `radon cc` reporta lo mismo: `calc - C (15)`, rango C (complejidad moderada-alta, riesgo medio). Se necesitan como máximo 15 caminos independientes.

4.3 Caminos independientes y casos de prueba

Camino base (CB-03): peso de tramo 1, urbana, normal, entre semana, sin cupón, sin flag. Cada camino siguiente cambia una decisión y agrega al menos una arista nueva.

#	Camino (nodos)	Arista nueva	Caso	Entradas (p, z, c, d, cup, flag)	Esperado según especificación
1	1-3-29	1→3	CB-01	0, urbana, normal, 1	Error "peso inválido"
2	1-2-3-29	2→3	CB-02	51, urbana, normal, 1	Error "peso inválido"
3	1-2-4-5-9-10-15-17-19-21-24-26-28-29	camino base	CB-03	1, urbana, normal, 1	$6.000
4	1-2-4-6-7-9-10-15-17-19-21-24-26-28-29	4→6, 6→7	CB-04	5, urbana, normal, 1	$9.000
5	1-2-4-6-8-9-10-15-17-19-21-24-26-28-29	6→8	CB-05	20, urbana, normal, 1	$17.000
6	1-2-4-5-9-11-12-15-17-19-21-24-26-28-29	9→11, 11→12	CB-06	1, rural, normal, 1	6.000 × 1,20 = $7.200
7	1-2-4-5-9-11-13-14-15-17-19-21-24-26-28-29	11→13, 13→14	CB-07	1, nacional, normal, 1	6.000 × 1,50 = $9.000
8	1-2-4-5-9-11-13-15-17-19-21-24-26-28-29	13→15	CB-08	1, internacional, normal, 1	Error (zona inválida)
9	1-2-4-5-9-10-15-16-17-19-21-24-26-28-29	15→16	CB-09	1, urbana, normal, 7	6.000 + 2.000 = $8.000
10	1-2-4-5-9-10-15-17-18-19-21-24-26-28-29	17→18	CB-10	1, urbana, frecuente, 1	6.000 × 0,95 = $5.700
11	1-2-4-5-9-10-15-17-19-20-21-24-26-28-29	19→20	CB-11	1, urbana, vip, 1	6.000 × 0,85 = $5.100
12	1-2-4-5-9-10-15-17-19-21-22-24-26-28-29	21→22, 22→24	CB-12	1, urbana, normal, 1, NOEXISTE	Error (cupón inexistente)

#	Camino (nodos)	Arista nueva	Caso	Entradas (p, z, c, d, cup, flag)	Esperado según especificación
13	1-2-4-5-9-10-15-17-19-21-22-23-24-26-28-29	22→23	CB-13	1, urbana, normal, 1, ENVIO10	6.000 × 0,90 = $5.400
14	1-2-4-5-9-10-15-17-19-20-21-22-23-24-25-26-28-29	24→25	CB-14	1, urbana, vip, 1, ENVIO10	4.590 → mínimo $5.000
15	1-2-4-5-9-10-15-17-19-21-24-26-27-28-29	26→27	CB-15	1, urbana, normal, 1, —, flag=True	$6.000 (e imprime el costo)

Observación: los caminos 8 y 12 existen en el código pero la especificación exige un error en esos casos (R2 y R7). El código heredado devuelve $6.000 en ambos, así que CB-08 y CB-12 fallan: son los defectos DEF-02 y DEF-05 vistos desde la estructura (falta una rama `else` que lance el error).

4.4 Cobertura de sentencias y de ramas

Cobertura de sentencias (100 %). Cada sentencia se ejecuta al menos una vez con estos casos:

Sentencia (nodo)	Caso que la ejecuta
3 `raise`	CB-01
5 `t = 6000`, 10 `t = t`	CB-10
7 `t = 9000`	CB-04
8 tramo 3	CB-05
12 recargo rural	CB-06
14 recargo nacional	CB-07
16 recargo fin de semana	CB-09
18 descuento frecuente	CB-10
20 descuento VIP, 23 cupón, 25 mínimo	CB-14
27 `print`	CB-15
28 `return`	todos menos CB-01 y CB-02

Conjunto para sentencias: **CB-01, CB-04, CB-05, CB-06, CB-07, CB-09, CB-10, CB-14 y CB-15** (9 casos).

Cobertura de ramas (100 %). Además de lo anterior hay que recorrer las salidas "falso" que no tienen sentencia propia:

Rama que falta	Caso que la recorre
2→3 (`p > 50` verdadero)	CB-02
13→15 (zona que no es ninguna de las tres)	CB-08
22→24 (cupón distinto de ENVIO10)	CB-12
21→24, 24→26, 15→17, 17→19, 19→21 (salidas "falso")	CB-03
22→23 con cliente normal	CB-13
19→20 sin cupón	CB-11

Conjunto para ramas: **los 15 casos CB-01 a CB-15** recorren las 42 aristas, es decir, las dos salidas de los 14 predicados. Los casos que solo aportan a ramas (y no a sentencias) son CB-02, CB-03, CB-08 y CB-12; CB-11 y CB-13

son redundantes para cobertura pero se mantienen porque completan el conjunto base de caminos.

Nota: `pytest-cov` mide ramas por línea, así que no distingue 1→3 de 2→3 (ambas están en la línea 12). Por eso CB-02 es necesario para la cobertura de condiciones aunque la herramienta ya marque la línea como cubierta con CB-01.

Parte 5 · Automatización, ejecución y defectos

5.1 Automatización

La suite está en tests/ y se ejecuta con pytest:

Archivo	Contenido	Pruebas
`tests/test_caja_negra.py`	CP-01 a CP-36 y tabla de decisión TD-01 a TD-12 (48 casos)	48
`tests/test_caja_blanca.py`	CB-01 a CB-15, un caso por camino independiente	15
Total		**63**

Comando: `python -m pytest -v --tb=no --cov=envios --cov-branch --cov-report=term-missing`

5.2 Ejecución contra el código heredado v1.0.0

Resultado real: **63 pruebas, 40 pasan y 23 fallan** (`evidencia/01-pruebas-v1.0.0.txt`).

Windows PowerShell - pruebas contra el codigo heredado v1.0.0

```
PS C:\Users\B6\Downloads\parcial> python -m pytest -v --tb=no --cov=envios --cov-branch --cov-report=term-missing
============================= test session starts =============================
platform win32 -- Python 3.13.14, pytest-9.2.0, pluggy-1.7.0 -- C:\Users\B6\AppData\Local\Programs\Python\Python313\python.exe
rootdir: C:\Users\B6\Downloads\parcial
plugins: cov-7.1.0
collecting ... collected 63 items

tests/test_caja_blanca.py::test_cb01_peso_cero PASSED                    [  1%]
tests/test_caja_blanca.py::test_cb02_peso_mayor_al_maximo PASSED         [  3%]
tests/test_caja_blanca.py::test_cb03_camino_base PASSED                  [  4%]
tests/test_caja_blanca.py::test_cb04_tramo_2 PASSED                      [  6%]
tests/test_caja_blanca.py::test_cb05_tramo_3 PASSED                      [  7%]
tests/test_caja_blanca.py::test_cb06_zona_rural PASSED                   [  9%]
tests/test_caja_blanca.py::test_cb07_zona_nacional PASSED                [ 11%]
tests/test_caja_blanca.py::test_cb08_zona_que_no_existe FAILED           [ 12%]
tests/test_caja_blanca.py::test_cb09_domingo PASSED                      [ 14%]
tests/test_caja_blanca.py::test_cb10_cliente_frecuente PASSED            [ 15%]
tests/test_caja_blanca.py::test_cb11_cliente_vip PASSED                  [ 17%]
tests/test_caja_blanca.py::test_cb12_cupon_que_no_es_envio10 FAILED      [ 19%]
tests/test_caja_blanca.py::test_cb13_cupon_envio10 PASSED                [ 20%]
tests/test_caja_blanca.py::test_cb14_costo_minimo PASSED                 [ 22%]
tests/test_caja_blanca.py::test_cb15_flag_imprime_el_costo PASSED        [ 23%]
tests/test_caja_negra.py::test_r1_peso_invalido[CP-01-0] PASSED          [ 25%]
tests/test_caja_negra.py::test_r1_peso_invalido[CP-02--0.01] PASSED      [ 26%]
tests/test_caja_negra.py::test_r1_peso_invalido[CP-05-50.01] PASSED      [ 28%]
tests/test_caja_negra.py::test_r1_peso_en_los_limites_validos[CP-03-0.01-6000] PASSED [ 30%]
tests/test_caja_negra.py::test_r1_peso_en_los_limites_validos[CP-04-50-41000] PASSED [ 31%]
tests/test_caja_negra.py::test_r1_peso_en_los_limites_validos[CP-06-49.99-41000] FAILED [ 33%]
tests/test_caja_negra.py::test_cp07_zona_invalida_es_error FAILED        [ 34%]
tests/test_caja_negra.py::test_r2_dia_fuera_de_rango_es_error[CP-08-0] FAILED [ 36%]
tests/test_caja_negra.py::test_r2_dia_fuera_de_rango_es_error[CP-09-8] FAILED [ 38%]
tests/test_caja_negra.py::test_r3_tarifa_base[CP-10-2.99-6000] PASSED    [ 39%]
tests/test_caja_negra.py::test_r3_tarifa_base[CP-11-3-9000] FAILED       [ 41%]
tests/test_caja_negra.py::test_r3_tarifa_base[CP-12-3.01-9000] PASSED    [ 42%]
tests/test_caja_negra.py::test_r3_tarifa_base[CP-13-9.99-9000] PASSED    [ 44%]
tests/test_caja_negra.py::test_r3_tarifa_base[CP-14-10-9000] PASSED      [ 46%]
tests/test_caja_negra.py::test_r3_tarifa_base[CP-15-10.01-9000] PASSED   [ 47%]
tests/test_caja_negra.py::test_r3_tarifa_base[CP-16-25-21000] PASSED     [ 49%]
tests/test_caja_negra.py::test_r4_recargo_por_zona[CP-17-urbana-9000] PASSED [ 50%]
tests/test_caja_negra.py::test_r4_recargo_por_zona[CP-18-rural-10800] PASSED [ 52%]
tests/test_caja_negra.py::test_r4_recargo_por_zona[CP-19-nacional-13500] PASSED [ 53%]
tests/test_caja_negra.py::test_r5_recargo_fin_de_semana[CP-20-5-9000] PASSED [ 55%]
tests/test_caja_negra.py::test_r5_recargo_fin_de_semana[CP-21-6-11000] FAILED [ 57%]
tests/test_caja_negra.py::test_r5_recargo_fin_de_semana[CP-22-7-11000] PASSED [ 58%]
tests/test_caja_negra.py::test_cp23_recargo_finde_va_antes_del_descuento FAILED [ 60%]
tests/test_caja_negra.py::test_cp24_cliente_frecuente PASSED             [ 61%]
tests/test_caja_negra.py::test_cp25_cliente_vip PASSED                   [ 63%]
tests/test_caja_negra.py::test_cp26_cupon_envio10 PASSED                 [ 65%]
tests/test_caja_negra.py::test_cp27_vip_y_cupon_se_aplican_ambos PASSED  [ 66%]
tests/test_caja_negra.py::test_cp28_frecuente_cupon_y_domingo PASSED     [ 68%]
tests/test_caja_negra.py::test_cp29_cupon_inexistente_es_error FAILED    [ 69%]
tests/test_caja_negra.py::test_cp30_cupon_en_minuscula_no_existe FAILED  [ 71%]
tests/test_caja_negra.py::test_cp31_costo_minimo PASSED                  [ 73%]
tests/test_caja_negra.py::test_cp32_los_50_redondean_hacia_arriba FAILED [ 74%]
tests/test_caja_negra.py::test_cp33_redondeo_hacia_arriba FAILED         [ 76%]
tests/test_caja_negra.py::test_cp34_redondeo_hacia_abajo PASSED          [ 77%]
tests/test_caja_negra.py::test_cp35_justo_sobre_el_minimo PASSED         [ 79%]
tests/test_caja_negra.py::test_cp36_caso_combinado PASSED                [ 80%]
tests/test_caja_negra.py::test_tabla_de_decision[TD-01-normal-None-1-9000] PASSED [ 82%]
tests/test_caja_negra.py::test_tabla_de_decision[TD-02-normal-None-6-11000] FAILED [ 84%]
tests/test_caja_negra.py::test_tabla_de_decision[TD-03-normal-ENVIO10-1-8100] PASSED [ 85%]
tests/test_caja_negra.py::test_tabla_de_decision[TD-04-normal-ENVIO10-6-9900] FAILED [ 87%]
tests/test_caja_negra.py::test_tabla_de_decision[TD-05-frecuente-None-1-8600] FAILED [ 88%]
tests/test_caja_negra.py::test_tabla_de_decision[TD-06-frecuente-None-6-10500] FAILED [ 90%]
tests/test_caja_negra.py::test_tabla_de_decision[TD-07-frecuente-ENVIO10-1-7700] FAILED [ 92%]
tests/test_caja_negra.py::test_tabla_de_decision[TD-08-frecuente-ENVIO10-6-9400] FAILED [ 93%]
tests/test_caja_negra.py::test_tabla_de_decision[TD-09-vip-None-1-7700] FAILED [ 95%]
tests/test_caja_negra.py::test_tabla_de_decision[TD-10-vip-None-6-9400] FAILED [ 96%]
tests/test_caja_negra.py::test_tabla_de_decision[TD-11-vip-ENVIO10-1-6900] FAILED [ 98%]
tests/test_caja_negra.py::test_tabla_de_decision[TD-12-vip-ENVIO10-6-8400] FAILED [100%]

============================== tests coverage ==============================
_____________ coverage: platform win32, python 3.13.14-final-0 _____________

Name        Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------
envios.py      29      0     26      0   100%
-----------------------------------------------------------
TOTAL          29      0     26      0   100%
========================== short test summary info ==========================
FAILED tests/test_caja_blanca.py::test_cb08_zona_que_no_existe - Failed: DID ...
FAILED tests/test_caja_blanca.py::test_cb12_cupon_que_no_es_envio10 - Failed:...
FAILED tests/test_caja_negra.py::test_r1_peso_en_los_limites_validos[CP-06-49.99-41000]
FAILED tests/test_caja_negra.py::test_cp07_zona_invalida_es_error - Failed: D...
```

```
FAILED tests/test_caja_negra.py::test_r2_dia_fuera_de_rango_es_error[CP-08-0]
FAILED tests/test_caja_negra.py::test_r2_dia_fuera_de_rango_es_error[CP-09-8]
FAILED tests/test_caja_negra.py::test_r3_tarifa_base[CP-11-3-9000] - Assertio...
FAILED tests/test_caja_negra.py::test_r5_recargo_fin_de_semana[CP-21-6-11000]
FAILED tests/test_caja_negra.py::test_cp23_recargo_finde_va_antes_del_descuento
FAILED tests/test_caja_negra.py::test_cp29_cupon_inexistente_es_error - Faile...
FAILED tests/test_caja_negra.py::test_cp30_cupon_en_minuscula_no_existe - Fai...
FAILED tests/test_caja_negra.py::test_cp32_los_50_redondean_hacia_arriba - As...
FAILED tests/test_caja_negra.py::test_cp33_redondeo_hacia_arriba - AssertionE...
FAILED tests/test_caja_negra.py::test_tabla_de_decision[TD-02-normal-None-6-11000]
FAILED tests/test_caja_negra.py::test_tabla_de_decision[TD-04-normal-ENVIO10-6-9900]
FAILED tests/test_caja_negra.py::test_tabla_de_decision[TD-05-frecuente-None-1-8600]
FAILED tests/test_caja_negra.py::test_tabla_de_decision[TD-06-frecuente-None-6-10500]
FAILED tests/test_caja_negra.py::test_tabla_de_decision[TD-07-frecuente-ENVIO10-1-7700]
FAILED tests/test_caja_negra.py::test_tabla_de_decision[TD-08-frecuente-ENVIO10-6-9400]
FAILED tests/test_caja_negra.py::test_tabla_de_decision[TD-09-vip-None-1-7700]
FAILED tests/test_caja_negra.py::test_tabla_de_decision[TD-10-vip-None-6-9400]
FAILED tests/test_caja_negra.py::test_tabla_de_decision[TD-11-vip-ENVIO10-1-6900]
FAILED tests/test_caja_negra.py::test_tabla_de_decision[TD-12-vip-ENVIO10-6-8400]
====================== 23 failed, 40 passed in 0.45s ======================
```

Cobertura medida con pytest-cov sobre `envios.py` (v1.0.0):

Sentencias	Ramas	Cobertura sentencias	Cobertura ramas
29 (0 sin cubrir)	26 (0 parciales)	**100 %**	**100 %**

Que la cobertura sea del 100 % no significa que el código sea correcto: la cobertura dice qué líneas se ejecutaron, no si el resultado es el que pide la especificación. Aquí las líneas con defectos están todas cubiertas y aun así devuelven valores errados.

5.3 Registro de defectos

Se analizó cada una de las 23 pruebas que fallan. Todas se explican por los 6 defectos de abajo. Las reglas de mi especificación que el código heredado **sí** cumple (y que por eso no se reportan) son: peso máximo 50 (coincide con PMAX = 50), tramo 2 hasta L2 inclusive, recargos rural 20 % y nacional 50 %, recargo de fin de semana antes de los descuentos (E par), aplicación de ambos descuentos en orden cliente → cupón (B impar) y mínimo de $5.000.

Escala de severidad: **Crítica** (pérdida masiva o caída), **Alta** (cobra un valor incorrecto en casos comunes), **Media** (cobra un valor incorrecto con entradas inválidas o poco frecuentes), **Baja** (cosmético). Prioridad: **P1** corregir ya, **P2** en este ciclo, **P3** después.

DEF-01 · El peso exacto de 3 kg se cobra como tramo 1

Campo	Detalle
Requisito	R3 (regla D impar: tramo 1 es peso < L1)
Línea	14: `if p <= CONFIG["L1"]`
Pasos	`calc(3, "urbana", "normal", 1)`
Esperado	$9.000 (3 no es menor que 3, va al tramo 2)
Obtenido	$6.000
Pruebas que fallan	CP-11
Severidad	**Alta**: cobra $3.000 menos en un valor de peso muy común (3 kg).
Prioridad	**P1**: afecta ingresos y es un cambio de una línea.

DEF-02 · Una zona inválida no genera error

Campo	Detalle
Requisito	R2
Línea	20 a 25: la cadena `if/elif` de zonas no tiene `else`
Pasos	`calc(5, "insular", "normal", 1)`
Esperado	Error por zona inválida
Obtenido	$9.000 (se trata como urbana sin avisar)
Pruebas que fallan	CP-07, CB-08
Severidad	**Media**: un error de digitación se cobra como urbana en silencio; solo ocurre con datos inválidos.
Prioridad	**P2**: debería validarse, pero normalmente la interfaz restringe las zonas.

DEF-03 · Un día fuera del rango 1 a 7 no genera error

Campo	Detalle
Requisito	R2
Línea	27: solo se pregunta `d > 6`, nunca se valida el rango
Pasos	`calc(5, "urbana", "normal", 0)` y `calc(5, "urbana", "normal", 8)`
Esperado	Error por día inválido en ambos
Obtenido	$9.000 con día 0 y $11.000 con día 8 (cobra fin de semana)
Pruebas que fallan	CP-08, CP-09
Severidad	**Media**: solo con datos inválidos, pero el día 8 llega a cobrar un recargo que no corresponde.
Prioridad	**P2**

DEF-04 · El sábado no cobra el recargo de fin de semana

Campo	Detalle
Requisito	R5 (sábado = 6 y domingo = 7)
Línea	27: `if d > 6`
Pasos	`calc(5, "urbana", "normal", 6)`
Esperado	$11.000 (9.000 + 2.000)
Obtenido	$9.000
Pruebas que fallan	CP-21, CP-23 y TD-02, TD-04, TD-06, TD-08, TD-10, TD-12
Severidad	**Alta**: se deja de cobrar $2.000 todos los sábados, es decir, a la mitad de los días de recargo.
Prioridad	**P1**: pérdida de ingresos recurrente.

DEF-05 · Un cupón inexistente se ignora

Campo	Detalle
Requisito	R7 (regla F impar: es error)
Línea	33 a 35: `if cup == "ENVIO10"` sin `else`
Pasos	`calc(5, "urbana", "normal", 1, "DESCUENTO99")`
Esperado	Error por cupón inexistente

Campo	Detalle
Obtenido	$9.000 (sin avisar que el cupón no sirvió)
Pruebas que fallan	CP-29, CP-30, CB-12
Severidad	**Media**: el cliente cree que tiene un descuento que no recibió, pero el sistema cobra el valor completo.
Prioridad	**P2**

DEF-06 · El redondeo trunca en vez de redondear al $100 más cercano

Campo	Detalle
Requisito	R8
Línea	40: `return int(t / 100) * 100`
Pasos	`calc(5, "urbana", "frecuente", 1)` (el valor calculado es 8.550)
Esperado	$8.600 (los $50 suben)
Obtenido	$8.500
Pruebas que fallan	CP-06 (40.992 da 40.900 en lugar de 41.000), CP-32, CP-33 y TD-05 a TD-12
Severidad	**Alta**: afecta a todo valor con descuento y siempre en contra del negocio (hasta $99 menos por envío).
Prioridad	**P1**: error sistemático en cada cotización con decimales.

Nota técnica: además del truncamiento, el cálculo en `float` puede dejar valores como 6884,999... en lugar de 6885, que redondearían mal aun con `round`. Por eso la corrección usa aritmética decimal exacta.

5.4 Matriz de trazabilidad con defectos

Requisito	Historia	Casos	Resultado v1.0.0	Defecto
R1	HU-1	CP-01 a CP-05, CB-01, CB-02	Pasan (CP-06 falla por DEF-06)	—
R2	HU-1	CP-07, CP-08, CP-09, CB-08	**Fallan**	DEF-02, DEF-03
R3	HU-2	CP-10 a CP-16	CP-11 **falla**	DEF-01
R4	HU-2	CP-17 a CP-19, CB-06, CB-07	Pasan	—
R5	HU-2	CP-20 a CP-23, TD-02, 04, 06, 08, 10, 12	CP-21, CP-23 y los TD con día 6 **fallan**	DEF-04
R6	HU-3	CP-24 a CP-28, CP-35, CP-36, CB-10, 11, 13	Pasan	—
R7	HU-1	CP-29, CP-30, CB-12	**Fallan**	DEF-05
R8	HU-3	CP-31 a CP-36, TD-05 a TD-12	CP-32, CP-33 y TD con valores en 50 **fallan**	DEF-06

5.5 Métricas sobre v1.0.0

Métrica	Valor
Casos ejecutados	63
Casos que pasan	40 → **63,5 %** (40 / 63)
Casos que fallan	23 → 36,5 %
Defectos por severidad	Crítica 0 · **Alta 3** (DEF-01, DEF-04, DEF-06) · **Media 3** (DEF-02, DEF-03, DEF-05) · Baja 0
Defectos por prioridad	P1 3 · P2 3
Líneas del código	40 (37 de código sin comentarios ni blancos, según radon)

Métrica	Valor
Densidad de defectos	6 defectos / 40 líneas × 100 = **15 defectos por cada 100 líneas** (16,2 sobre las 37 líneas de código)
Cobertura de sentencias	**100 %** (29 de 29)
Cobertura de ramas	**100 %** (26 de 26)
Complejidad ciclomática	15

Parte 6 · Mantenimiento y refactorización

6.1 Malos olores del código heredado v1.0.0

Las líneas son las de `envios.py` en v1.0.0 (`git show v1.0.0:envios.py`).

#	Mal olor (nombre técnico)	Línea(s)	Deuda técnica que genera
1	**Números mágicos** (*Magic Numbers*): 50, 6000, 9000, 800, 0.05, 0.10, 5000, 100	12, 15, 17, 19, 30, 35, 37, 40	Quien lee no sabe qué significa cada número. Un cambio de tarifa obliga a buscar y editar valores sueltos, con riesgo de dejar alguno sin cambiar. Además la mitad de los valores está en `CONFIG` y la otra mitad en el código, sin criterio.
2	**Argumento bandera** (*Flag Argument*) y mezcla de lógica con entrada/salida	11, 38, 39	La función hace dos cosas según un booleano e imprime en consola desde la lógica de negocio. Es difícil de probar y de reutilizar en un servicio web. Cada llamada puede comportarse distinto.
3	**Nombres misteriosos** (*Mystery Name*): `p`, `z`, `c`, `d`, `cup`, `t`	11 a 40	El código no se explica solo; hay que leer toda la función para saber que `t` es el valor acumulado. Cuesta mucho más el mantenimiento por alguien nuevo.
4	**Método largo / función Dios** (*Long Method*, *God Function*)	11 a 40	Una sola función valida, calcula tarifa, recargos, descuentos y redondea. Complejidad ciclomática 15, difícil de probar por partes. Cualquier cambio toca todo.
5	**Condicional en cadena sobre un tipo** (*Switch Statements* / *Conditional Complexity*)	20 a 25	Cada zona nueva obliga a modificar la función con otro `elif` (viola el principio abierto/cerrado). Es justo lo que se resuelve con el patrón Estrategia.
6	**Código muerto / sentencia redundante** (*Dead Code*): `t = t`	21	No hace nada, confunde al lector y sugiere que algo falta.
7	**Código duplicado** (*Duplicated Code*): `t = t ± t * x` repetido en cinco lugares	23, 25, 30, 32, 35	Si cambia la forma de aplicar un porcentaje, hay que corregirla cinco veces.
8	**Condicionales anidados y comparación no idiomática** (`!= None` con `if` dentro de otro `if`)	33 a 35	Es más difícil de leer y de seguir; en Python debe ser `is not None`. Y la rama `else` faltante ocultó el defecto DEF-05.
9	**Obsesión por los primitivos** (*Primitive Obsession*): zona, cliente y cupón son cadenas sueltas sin validación	20, 29, 31, 34	Un error de escritura (“Rural”, “rural ”) pasa en silencio. Origen de DEF-02 y DEF-05.
10	**Uso de `float` para dinero**	19 a 40	Los errores de precisión (6884,999…) pueden alterar el redondeo. Se agrava con el defecto DEF-06.
11	**Estado global mutable** (`CONFIG` como diccionario modificable)	2 a 9	Cualquier módulo puede cambiar las tarifas en ejecución y las pruebas se contaminan entre sí.

6.2 Clasificación de los cambios

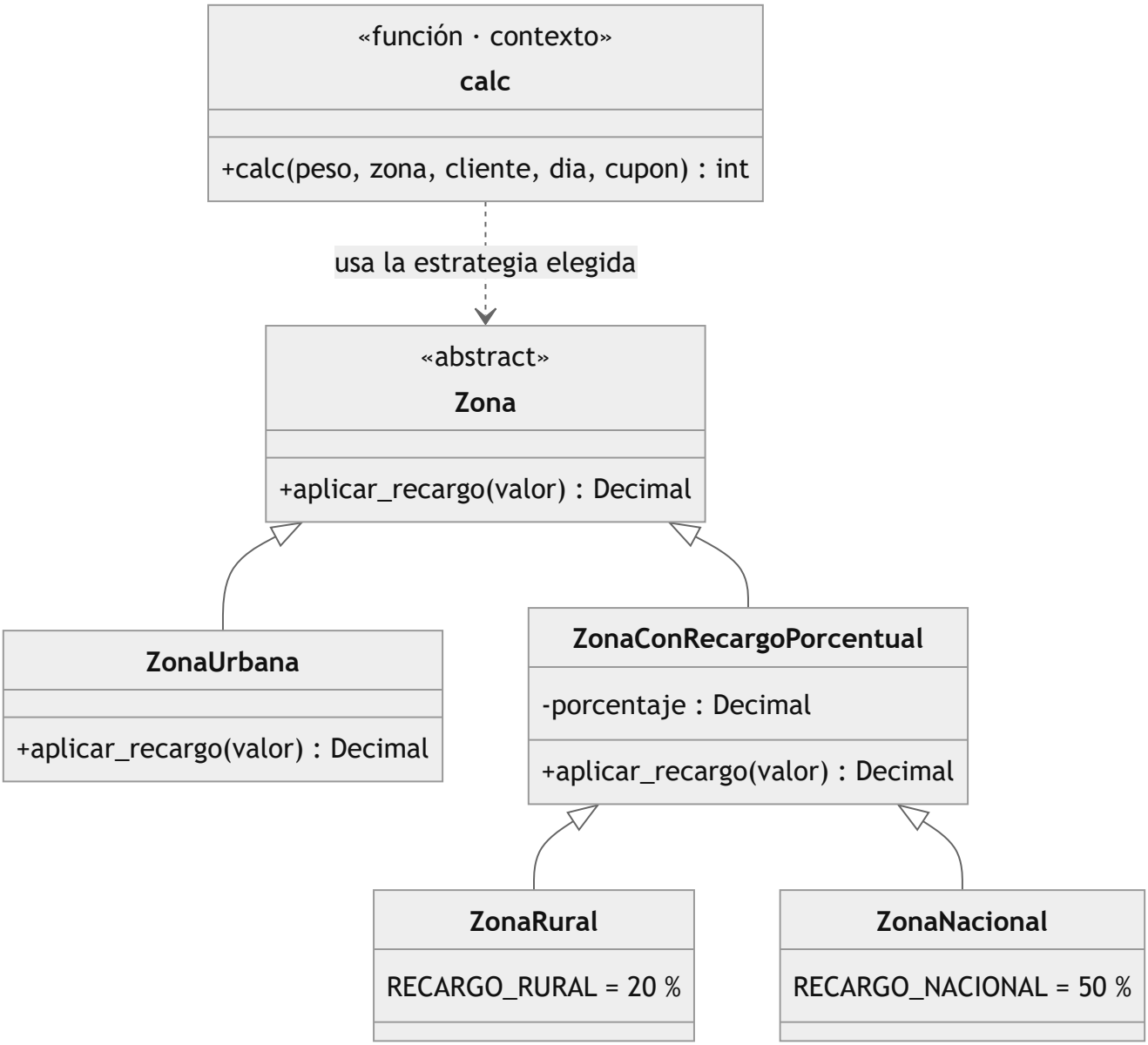
Cambio	Tipo de mantenimiento	Justificación
DEF-01 a DEF-06 (corregir límite L1, zona, día, sábado, cupón, redondeo)	**Correctivo**	Se corrige un comportamiento que incumple la especificación.

Cambio	Tipo de mantenimiento	Justificación
Usar `Decimal` en lugar de `float`	**Correctivo**	Parte de la corrección de DEF-06 (precisión del redondeo).
Extraer funciones, renombrar variables, eliminar `t = t` y duplicación	**Preventivo**	No cambia el comportamiento; reduce el riesgo de futuros defectos y facilita el cambio.
Reemplazar números mágicos por constantes con nombre	**Preventivo**	Igual: reduce la deuda técnica sin cambiar resultados.
Aplicar el patrón Estrategia a las zonas	**Preventivo**	Prepara el código para zonas nuevas (como la insular de la Parte 8).
Eliminar el argumento `flag`	**Perfectivo**	Mejora la interfaz de la función; es un cambio incompatible, por eso sube la versión mayor.
Pruebas nuevas de unidad y de regresión	**Preventivo**	Protegen el código frente a cambios futuros.
Zona insular (Parte 8)	**Perfectivo**	Amplía la funcionalidad por una solicitud del negocio. No es adaptativo porque no responde a un cambio del entorno técnico o legal (sistema operativo, ley, plataforma).

6.3 Refactorización realizada (rama `refactor/estrategia`)

1. **Funciones extraídas:** `validar_peso`, `obtener_zona`, `validar_dia`, `obtener_descuento_cupon`, `tarifa_base`, `es_fin_de_semana`, `aplicar_descuento`, `aplicar_costo_minimo`, `redondear`. `calc` queda como una receta de 10 pasos.
2. **Constantes con nombre:** `PESO_MAXIMO`, `LIMITE_TRAMO_1`, `LIMITE_TRAMO_2`, `TARIFA_TRAMO_1`, `TARIFA_TRAMO_2`, `TARIFA_POR_KG_EXTRA`, `RECARGO_RURAL`, `RECARGO_NACIONAL`, `RECARGO_FIN_DE_SEMANA`, `DESCUENTO_FRECUENTE`, `DESCUENTO_VIP`, `DESCUENTO_ENVIO10`, `COSTO_MINIMO`, `MULTIPLO_REDONDEO`, entre otras.
3. **Argumento `flag` eliminado**, junto con el `print`. Quien necesite el valor lo imprime en el código que llama a `calc`.
4. **Patrón Estrategia aplicado a las zonas:** cada zona es una clase que implementa `aplicar_recargo`. `calc` ya no conoce las zonas: pide la estrategia y la usa.
5. **Nombres claros:** `peso`, `zona`, `cliente`, `dia`, `cupon`, `valor`.

Diagrama de clases UML



El diccionario `ZONAS` relaciona el nombre de la zona con su estrategia. Agregar una zona nueva es crear una clase y registrarla; `calc` no cambia.

6.4 Corrección de defectos

Los seis defectos se corrigieron cada uno en su rama (`fix/DEF-01` a `fix/DEF-06`), ver la Parte 7. La versión resultante se etiquetó como `v1.0.1`.

6.5 Pruebas de regresión

Se ejecutaron las mismas pruebas (63 originales) tras las correcciones y tras la refactorización, y se sumaron 23 pruebas de unidad de las piezas nuevas.

Versión	Resultado	Evidencia
v1.0.0 (heredado)	23 fallan, 40 pasan (de 63)	`evidencia/01-pruebas-v1.0.0.png`
v1.0.1 (defectos corregidos)	**63 pasan**	`evidencia/02-pruebas-v1.0.1.png`
v2.0.0 (refactorizado)	**85 pasan, 1 omitida** (de 86)	`evidencia/03-pruebas-v2.0.0.png`

La prueba omitida es CB-15 (verifica que `flag=True` imprima): no aplica porque el argumento `flag` ya no existe. Se omite con una condición explícita y no se borra, para que quede registro de la decisión. Las otras 62 pruebas originales pasan sin modificar ni una sola.

 Regresión en verde, v1.0.1

 Regresión en verde, v2.0.0

6.6 Antes y después

Medida	v1.0.0	v2.0.0
Complejidad ciclomática de `calc`	**15** (rango C)	**2** (rango A)
Complejidad máxima de cualquier función	15	3
Complejidad promedio (radon)	15	1,7
Funciones / clases	1 / 0	10 / 5
Líneas totales	40	156
Líneas de código (sin comentarios ni blancos)	37	96
Argumentos de `calc`	6 (con `flag`)	5
Cobertura de sentencias	100 % (29 de 29)	100 % (87 de 87)
Cobertura de ramas	100 % (26 de 26)	100 % (12 de 12)
Defectos conocidos abiertos	6	0

Lectura honesta: **el código creció** (de 40 a 156 líneas) por las constantes con nombre, las clases de zona y los docstrings. Lo que se ganó no es brevedad: la complejidad de cada pieza bajó de 15 a 3 como máximo, cada regla vive en un solo lugar y se puede probar por separado. Por eso el número de líneas no es un buen indicador de calidad, y la cobertura del 100 % ya era alta antes de corregir (no garantizaba que los resultados fueran correctos).

Parte 7 · Gestión de configuración

Repositorio: https://github.com/calo159/EnviaYa

7.1 Etiqueta del código heredado

El commit inicial contiene `envios.py` tal como se entregó, con solo el `CONFIG` diligenciado con mis valores, y está etiquetado `v1.0.0`.

7.2 Ramas

Una rama por cada defecto, una de refactorización, una de pruebas y una de la solicitud de cambio. Todas se integraron a `main` con `git merge --no-ff`, para que el historial conserve la rama de cada trabajo.

Rama	Contenido	Integración
`test/suite`	48 casos de caja negra y 15 de caja blanca, automatizados con pytest	Merge a `main`. Se integró antes de las correcciones, por eso contra v1.0.0 la suite falla (23 rojos).
`fix/DEF-01`	El tramo 1 no incluye L1 (R3)	Merge a `main`
`fix/DEF-02`	Zona inválida lanza error (R2)	Merge a `main`
`fix/DEF-03`	Día fuera de 1 a 7 lanza error (R2)	Merge a `main`
`fix/DEF-04`	El sábado cobra el recargo de fin de semana (R5)	Merge a `main`
`fix/DEF-05`	Cupón inexistente lanza error (R7)	Merge a `main`
`fix/DEF-06`	Redondeo al $100 más cercano, los $50 suben (R8)	Merge a `main` → etiqueta `v1.0.1`
`refactor/estrategia`	Funciones extraídas, constantes, sin `flag`, patrón Estrategia	Merge a `main` → etiqueta `v2.0.0`
`feature/zona-insular`	Solicitud de cambio 0: pruebas primero y luego implementación	Merge a `main` → etiqueta `v2.1.0`

Orden de trabajo: cada rama de defecto se creó desde el `main` ya actualizado con la anterior, así que las integraciones no generaron conflictos. Tras cada corrección se ejecutó la suite completa: los rojos bajaron 23 → 22 → 20 → 18 → 13 → 10 → 0.

7.3 Versionado semántico

El formato es `MAJOR.MINOR.PATCH`: major si se rompe la interfaz, minor si se agrega funcionalidad compatible, patch si solo se corrigen defectos.

Versión	Qué cambió	Por qué esa versión
`v1.0.0`	Código heredado	Punto de partida.
`v1.0.1`	Se corrigen DEF-01 a DEF-06	**Patch**: solo correcciones, la firma de `calc` es la misma y no se agrega funcionalidad.
`v2.0.0`	Refactorización y eliminación de `flag`	**Major**: ver abajo.

Versión	Qué cambió	Por qué esa versión
`v2.1.0`	Zona insular	**Minor**: funcionalidad nueva que no rompe a quien ya usa la función.

¿Quitar `flag` hace que sea major? Sí. `flag` era parte de la interfaz pública de `calc`. Quien llamaba `calc(5, "urbana", "normal", 1, None, True)` ahora recibe un `TypeError` porque la función ya no acepta un sexto argumento, y además deja de imprimir en consola. Eso es un cambio incompatible hacia atrás, y el versionado semántico exige subir el major aunque el resto de la función siga igual. Matiz: quien llamaba sin `flag` (el caso normal) no nota la diferencia en la firma; el major es por quienes sí lo usaban, ya que el versionado semántico protege a todos los usuarios de la interfaz.

Las correcciones de defectos (v1.0.1) cambian resultados (por ejemplo, el sábado ahora cuesta $2.000 más), pero no son un cambio incompatible: el comportamiento anterior era un error contra la especificación, así que se publican como patch.

7.4 CHANGELOG

Ver CHANGELOG.md en el formato *Keep a Changelog*.

7.5 Notas de la versión v2.1.0

EnvíaYa · cálculo de costos v2.1.0

Novedades - Nueva zona **insular** (San Andrés) con recargo del 70 %. Los envíos insulares no aceptan cupón.

Si vienes de v1.x - `calc` ya no recibe el argumento `flag` ni imprime nada. Actualiza las llamadas: `calc(peso, zona, cliente, dia, cupon=None)`. - Un día fuera de 1 a 7, una zona inválida o un cupón inexistente ahora lanzan `ValueError` en lugar de calcularse o ignorarse. - El sábado ahora cobra el recargo de fin de semana. - El tramo 1 es peso < 3 kg (antes 3 kg caía en el tramo 1). - El resultado se redondea al múltiplo de $100 más cercano (antes se truncaba).

Calidad - 100 pruebas automatizadas pasan y 1 se omite (la del `flag`). Cobertura de sentencias y de ramas: 100 %. - Complejidad ciclomática máxima por función: 3.

7.6 Historial de Git

La salida de `git log --oneline --graph --all` está en evidencia/07-git-log.txt y al final del informe. Los commits están hechos con la hora real en que se trabajó cada parte: el primero a las 16:34 y los siguientes a medida que se completaba cada parte, sin alterar las fechas.

Parte 8 · Solicitud de cambio y análisis de impacto

Mi solicitud: (A + F) mod 3 = (4 + 5) mod 3 = 0 → *Nueva zona insular (San Andrés) con un recargo del 70 %. Los envíos insulares no aceptan cupón.*

8.1 Análisis de impacto con la matriz de trazabilidad

Elemento	¿Se afecta?	Detalle
R1 peso válido	No	Se mantiene igual para la nueva zona.
R2 zona válida	**Sí**	"insular" deja de ser una zona inválida: pasa a ser la cuarta zona válida.
R3 tarifa base	No	La base no depende de la zona.
R4 recargo por zona	**Sí**	Se agrega: insular +70 %.
R5 fin de semana	No	Se mantiene: se suma antes de los descuentos, también en la zona insular.
R6 descuentos	**Sí, parcial**	Cliente frecuente y VIP siguen aplicando; el cupón ENVIO10 no aplica en la zona insular.
R7 cupón inexistente	**Sí, parcial**	En zona insular, cualquier cupón (válido o no) es error.
R8 mínimo y redondeo	No	
R9 (nuevo)	**Nuevo**	Los envíos insulares no aceptan cupón.

Interpretación adoptada: "no aceptan cupón" significa que, si en un envío insular llega un cupón, se lanza error (`los envios insulares no aceptan cupon`). Se eligió error y no ignorarlo en silencio para ser coherente con mi regla F impar y porque el cliente no debe creer que recibió un descuento que no se aplicó. Es una pregunta abierta para el negocio.

Matriz de trazabilidad afectada

Requisito	Historia	Casos existentes afectados	Casos nuevos	Parte del código
R2	HU-1	**CP-07** y la prueba unitaria `obtener_zona` usaban "insular" como zona inválida; ahora usan "internacional"	CI-13	`ZONAS` y `obtener_zona`
R4	HU-2	CP-17 a CP-19 siguen igual (regresión)	CI-01 a CI-04	Nueva clase `ZonaInsular` y constante `RECARGO_INSULAR`
R5	HU-2	Sin cambios	CI-05, CI-07, CI-08	Sin cambios
R6	HU-3	CP-24 a CP-28 y TD-01 a TD-12 siguen igual (regresión)	CI-06, CI-07, CI-08, CI-11	`calc` (validación de cupón)
R7	HU-1	CP-29 y CP-30 siguen igual	CI-10	`calc`
R9 (nuevo)	HU-1 (criterio nuevo)	—	CI-09, CI-10, CI-11	`Zona.acepta_cupon`, `calc`

Historia afectada: HU-1 (validación) y HU-2 (recargos) ganan criterios nuevos:

- *Dado* una zona insular y 5 kg, *cuando* calculo, *entonces* se suma 70 % a la base ($9.000 → $15.300).
- *Dado* una zona insular y un cupón, *cuando* calculo, *entonces* se lanza error.

Partes del código que cambian (todas en `envios.py`): constante `RECARGO_INSULAR`; atributo `acepta_cupon` en `Zona`; clase `ZonaInsular`; entrada `"insular"` en `ZONAS`; y una validación en `calc`. Gracias al patrón Estrategia de la Parte 6, `tarifa_base`, `redondear` y las demás funciones no se tocan, y las otras tres zonas tampoco.

Riesgos del cambio: (1) que la zona insular rompa las zonas existentes → se mitiga con la regresión completa; (2) que un caso que daba error ahora no lo dé (CP-07) → se actualizó de forma explícita; (3) interpretación de "no aceptan cupón" → documentada.

8.2 Tipo de mantenimiento

Perfectivo (evolutivo): amplía la funcionalidad por una solicitud del negocio. No es correctivo (no hay defecto), ni adaptativo (no cambia el entorno técnico o legal), ni preventivo.

8.3 Implementación con pruebas primero

Rama `feature/zona-insular`, en este orden:

1. **Commit de pruebas (rojo):** 15 pruebas nuevas en `tests/test_zona_insular.py` (CI-01 a CI-13) y la actualización de CP-07. Se ejecutaron antes de tocar el código: **10 fallan, 5 pasan** (las 5 que ya pasaban son las que verifican que lo existente no cambia o que ya daban error).
2. **Commit de implementación (verde):** `ZonaInsular` y la validación de cupón. Resultado: **100 pasan, 1 omitida**, cobertura 100 % de sentencias y ramas.

 Pruebas de la zona insular fallando antes de implementar

 Suite completa en verde tras implementar

Cálculo a mano de dos casos nuevos:

CI-08 · peso = 3 kg, insular, vip, día 6, sin cupón 1. Tramo: 3 no es menor que 3, tramo 2 → $9.000 2. Zona insular +70 % → 9.000 × 1,70 = $15.300 3. Sábado, recargo antes de los descuentos → 15.300 + 2.000 = $17.300 4. VIP 15 % → 17.300 × 0,85 = $14.705 5. Se redondea a $100 (5 < 50, baja) → **$14.700**

CI-07 · peso = 5 kg, insular, frecuente, día 7, sin cupón 1. Tramo 2 → $9.000 → insular: $15.300 2. Domingo: 15.300 + 2.000 = $17.300 3. Frecuente 5 % → 17.300 × 0,95 = $16.435 4. Se redondea a $100 (35 < 50, baja) → **$16.400**

8.4 Nueva versión

v2.1.0. Se agrega una función nueva sin romper la interfaz de `calc` (misma firma), así que es un cambio *minor* según el versionado semántico. Hay un matiz: un envío insular que antes daba error ahora se calcula; se considera parte de la nueva funcionalidad y no una ruptura de la interfaz, porque lo que se agrega es un valor aceptado, no una firma distinta.

Anexo · Salida de git log --oneline --graph --all

```
* 6094fea (HEAD -> main) docs: gestion de configuracion, versionado semantico y notas de version (Parte 7); README
*   6bb3b6a (tag: v2.1.0) Merge branch 'feature/zona-insular': nueva zona insular
|\
| * 8cc6b3b (feature/zona-insular) docs: analisis de impacto de la zona insular (Parte 8) y CHANGELOG 2.1.0
| * e4fa324 feat: zona insular (San Andres) con recargo del 70 % y sin cupones
| * 8b8ea0e test: pruebas de la zona insular escritas primero (rojo); CP-07 usa otra zona invalida
|/
* 20f61a2 (tag: v2.0.0) docs: malos olores, clasificacion, UML, regresion y comparacion (Parte 6) y CHANGELOG
*   8224a90 Merge branch 'refactor/estrategia': refactorizacion con patron Estrategia
|\
| * 3bbe5df (refactor/estrategia) test: pruebas de unidad de las funciones y estrategias extraidas
| * 066579f refactor: extrae funciones, constantes con nombre, quita flag y aplica Estrategia a las zonas
|/
* c8ef56a test: evidencia de regresion en verde tras las correcciones (v1.0.1)
*   0593ea4 (tag: v1.0.1) Merge branch 'fix/DEF-06'
|\
| * 6d5e766 (fix/DEF-06) fix(DEF-06): redondea al multiplo de 100 mas cercano, los 50 suben (R8)
|/
*   61b64cf Merge branch 'fix/DEF-05'
|\
| * 2dbfedd (fix/DEF-05) fix(DEF-05): un cupon inexistente lanza error (R7)
|/
*   1fec9f2 Merge branch 'fix/DEF-04'
|\
| * a40ce54 (fix/DEF-04) fix(DEF-04): el sabado (dia 6) tambien cobra el recargo de fin de semana (R5)
|/
*   23b947e Merge branch 'fix/DEF-03'
|\
| * 2513bae (fix/DEF-03) fix(DEF-03): un dia fuera de 1 a 7 lanza error (R2)
|/
*   93ee641 Merge branch 'fix/DEF-02'
|\
| * 0f82cee (fix/DEF-02) fix(DEF-02): una zona invalida lanza error (R2)
|/
*   14761f3 Merge branch 'fix/DEF-01'
|\
| * 637b9b5 (fix/DEF-01) fix(DEF-01): el tramo 1 no incluye el limite L1 (peso < 3)
|/
* 52c3ecc docs: ejecucion contra v1.0.0, cobertura, registro de defectos y metricas (Parte 5)
* 0c4d004 docs: grafo de flujo, complejidad ciclomatica y caminos (Parte 4)
*   efd7f3e Merge branch 'test/suite': suite automatizada de caja negra y caja blanca
|\
| * 32e6d78 (test/suite) test: casos de caja blanca, un caso por camino independiente
| * 0184e7a test: automatiza los 48 casos de caja negra con pytest
|/
* 7733dfb docs: particiones, valores limite, tabla de decision y matriz de casos (Parte 3)
* 3c0d169 docs: plan de pruebas con niveles, riesgos y criterios (Parte 2)
* 0cf003b docs: parametros, especificacion R1-R8 e historias de usuario (Parte 1)
* 78594fd (tag: v1.0.0) feat: codigo heredado de EnviaYa con CONFIG personalizado
```